

Des ABR aux AVL

BUT Informatique - S5 - Qualité algorithmique

1 Arbres binaires de recherche

Un *arbre binaire de recherche* (ABR) est un arbre binaire dont chaque noeud possède une clé et qui satisfait la propriété suivante: soit x un noeud, si y est un noeud du sous-arbre gauche de x , alors $y.cle \leq x.cle$; si y est noeud du sous-arbre droit de x , alors $y.cle \geq x.cle$.



Figure 1: Exemples d'ABR

Question 1

Donner l'algorithme d'insertion $\text{INSERER}(a, v)$ d'une valeur v dans l'ABR a . On notera NIL l'arbre vide.

SOLUTION

```
function INSERER( $a, v$ )
    pere  $\leftarrow NIL$ 
     $x \leftarrow a$ 
    while  $x \neq NIL$  do
        pere  $\leftarrow x$ 
        if  $v < x.cle$  then
             $x \leftarrow x.gauche$ 
        else
             $x \leftarrow x.droite$ 
    if pere ==  $NIL$  then
        return feuille( $v$ )
    else if  $v < pere.cle$  then
        pere.gauche  $\leftarrow$  feuille( $v$ )
    else
        pere.droite  $\leftarrow$  feuille( $v$ )
    return a
```

Question 2

Quelle est la complexité de cette fonction dans le pire cas ?

L'insertion parcourt l'ABR de la racine à une feuille. Dans le pire cas, il parcourt la plus longue branche qui, dans le cas d'un ABR "peigne" à n noeud sera de longueur $n - 1$. Donc, la complexité dans le pire cas est $O(n)$. Si on a la garantie que l'arbre est de hauteur minimale (arbre complet), alors la complexité dans le pire cas est de $O(\log n)$.

2 Arbres de Fibonacci

On rappelle que la hauteur $h(a)$ d'un arbre a est définie par induction ainsi:

- $h(NIL) = -1$
- $h(a) = 1 + \max(h(a.gauche), h(a.droite))$

Un arbre a à n noeuds est dit *équilibré*, s'il est de hauteur minimale, c'est-à-dire tel que $h(a) = O(\log(n))$.

Le *déséquilibre* d'un arbre est défini comme étant la différence de hauteur entre ses sous-arbres gauche et droit:

$$deseq(a) = h(a.gauche) - h(a.droite)$$

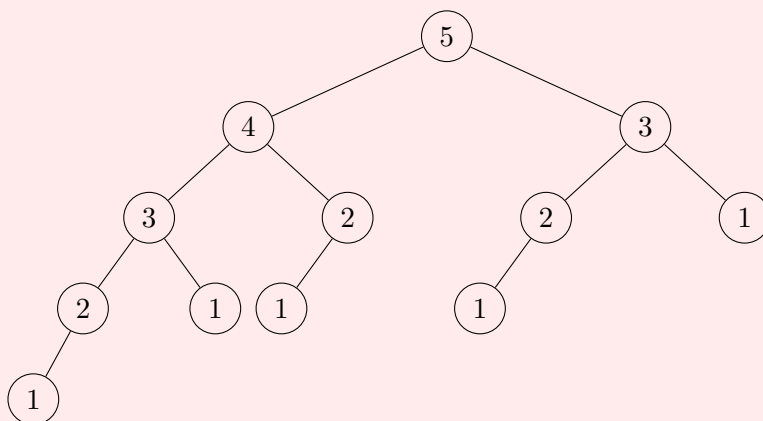
Question 3

Un arbre de Fibonacci d'ordre p est:

- l'arbre vide si $p = 0$;
- un arbre réduit à une feuille si $p = 1$;
- un arbre dont le fils gauche est un arbre de Fibonacci d'ordre $p - 1$ et le fils droit un arbre de Fibonacci d'ordre $p - 2$ si $p > 2$.

Dessiner le squelette des arbres de Fibonacci d'ordres 0 à 5. Combien un arbre de Fibonacci d'ordre p possède-t-il de feuilles ? Quelle est sa hauteur ? Calculer son déséquilibre en tout noeud.

Squelette d'un arbre de Fibonacci d'ordre 5 (en indiquant l'ordre dans les noeuds):



Si f_p désigne le nombre de feuilles d'un arbre de Fibonacci d'ordre p alors

$$f_0 = 0, f_1 = 1 \text{ et } \forall p \geq 2, f_p = f_{p-1} + f_{p-2}$$

for f_p est égal au p ième nombre de Fibonacci.

Montrons que la hauteur d'un arbre de Fibonacci d'ordre p est égal à $p - 1$ (pour tout $p \geq 1$).

- C'est clair si $p = 1$ ou $p = 2$.
- Si $p > 2$, supposons le résultat acquis aux rangs $p-1$ et $p-2$, et considérons un arbre de Fibonacci a d'ordre p . Par construction, $h(a) = 1 + \max(p-2, p-3) = p-1$, ce qui prouve le résultat au rang p .

Montrons maintenant par récurrence sur $p \in \mathbb{N}$ que tout nœud interne d'un arbre de Fibonacci d'ordre p a un déséquilibre égal à 1.

- C'est clair pour $p = 0$ et $p = 1$ puisqu'il n'y a pas de nœud interne dans ces deux arbres pour lesquels le déséquilibre est 0.
- Si $p > 2$, supposons le résultat acquis aux rangs $p-1$ et $p-2$. Par construction, tout nœud interne appartenant au fils gauche ou au fils droit a un déséquilibre égal à 0. Il reste à examiner le cas de la racine. Or nous venons de prouver que son fils gauche a une hauteur égale à $p-2$ et son fils droit à $p-3$ donc son déséquilibre est égal à $(p-2) - (p-3) = 1$, ce qui achève de prouver le résultat annoncé.

Question 4

Écrire une fonction `ORDRE(a)` qui renvoie -1 si l'arbre a n'est pas un arbre de Fibonacci et l'ordre p de cette arbre sinon.

SOLUTION

```
function ORDRE(a)
  if a == NIL ou estFeuille(a) then
    return 0
  else
    ordreGauche = ordre(a.gauche)
    ordreDroit = ordre(a.droite)
    if (ordreGauche == -1) or (ordreDroit == -1) or (ordreGauche - ordreDroit != 1) then
      return -1
    else
      return ordreGauche + 1
```

Question 5

Quelle est le coût de cette fonction ?

SOLUTION

On parcourt tous les nœuds et pour chaque nœud on effectue des opérations en $O(1)$. Donc, le coût total est $O(n)$ où n est le nombre de nœuds de l'arbre.

Question 6

Montrer que le nombre de nœuds d'un arbre de Fibonacci d'ordre p est $N_p = F_{p+3} - 1$ (où F_n désigne le n -ième nombre de Fibonacci).

SOLUTION

On procède par induction sur p .

- Si $p = 0$ le nombre de nœuds est $N_0 = 0$ qui est bien égal à $F_3 - 1 = 1 - 1 = 0$. Idem pour $p = 1$...

- Hypothèse d'induction: on suppose qu'il existe un entier k tel que pour tout $p \geq k$ on a $N_p = F_{p+3} - 1$.
- Montrons la propriété au rang $k + 1$, c'est-à-dire $N_{k+1} = F_{k+4} - 1$. Le nombre de noeuds d'un arbre de Fibonacci d'ordre $k + 1$ est, par construction, $N_{k+1} = N_k + N_{k-1} + 1$. Or, par hypothèse d'induction, $N_k = F_{k+3} - 1$ et $N_{k-1} = F_{k+2} - 1$, par conséquent $N_{k+1} = F_{k+3} + F_{k+2} - 1$. Aussi, par définition de la suite de Fibonacci, $F_{k+3} + F_{k+2} = F_{k+4}$, ce qui montre que $N_{k+1} = F_{k+4} - 1$.

Question 7

La suite de Fibonacci croît de façon exponentielle. En effet, pour de grandes valeurs de h , on a l'équivalence:

$$F_h \approx \frac{\varphi^h}{\sqrt{5}}$$

où φ est le nombre d'or, soit $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$. En admettant cette équivalence, montrer que tout arbre de Fibonacci est équilibré.

SOLUTION

Nous avons vu que le nombre de noeuds n d'un arbre de Fibonacci d'ordre p est $n = F_{p+3} - 1 = F_{h+2} - 1$ où h est la hauteur de l'arbre (puisque $h = p - 1$). Par conséquent, $n \geq c\varphi^h$. En appliquant le logarithme de chaque côté de l'inégalité, on obtient:

$$\log(n) \geq h \log(\varphi) + \log(c)$$

et donc

$$h \leq C \log(n)$$

où C est une constante qui dépend de φ et c . Ceci montre que la hauteur d'un arbre de Fibonacci est $O(\log(n))$.

Question 8

Soit a un arbre binaire de recherche ayant une structure de Fibonacci, quel est le coût, dans le pire cas, d'une recherche dans cet arbre ?

SOLUTION

La recherche dans un arbre de binaire de recherche a a un coût $O(h(a))$. Or, les arbres de Fibonacci à n noeuds ont une hauteur $O(\log(n))$, par conséquent, le coût d'une recherche dans un ABR Fibonacci a un coût $O(\log(n))$.

3 Arbres AVL

Les arbres de Fibonacci sont à priori de bons candidats pour organiser de façon optimale des ABR qui garantissent des recherches en $O(\log(n))$. Cependant, leur structure est trop rigide car il ne peut y avoir d'algorithme d'insertion qui préserve la structure de Fibonacci. En effet, par exemple l'insertion d'un élément dans un arbre d'ordre 2 (ie. avec 2 noeuds) devrait renvoyer un arbre de Fibonacci à 3 noeuds, or il n'existe pas de tel arbre de Fibonacci... Nous allons introduire un autre type d'arbre, les *arbres AVL*, semblables aux arbres de Fibonacci mais moins rigide.

Un arbre binaire de recherche a est un *arbre AVL* (du nom de leurs auteurs Georgii Adelson-Velsky et Evgueni Landis) ssi a est l'arbre vide ou

⁰Exercices inspirés de Jean-Pierre Becirspahic

- $a.gauche$ et $a.droite$ sont des arbres AVL, et
- $deseq(a) \in \{-1, 0, 1\}$

Autrement dit, dans un arbre AVL tous les noeuds possèdent des sous-arbres gauche et droite dont les hauteurs diffèrent d'au plus 1.

Question 9

Tous les ABR ayant une structure de Fibonacci sont-ils des AVL ? Est-ce que tous les AVL ont une structure de Fibonacci ?

SOLUTION

Bien sûr, tout arbre de Fibonacci est un AVL puisque nous avons vu que tous leurs noeuds ont un déséquilibre de 1. L'inverse est faux: par exemple, un ABR complet est un AVL (tous les déséquilibres sont nuls) mais ce n'est pas un arbre de Fibonacci.

Question 10

Les arbres AVL sont-ils équilibrés ? Pourquoi ?

SOLUTION

Oui, les arbres AVL sont équilibrés. En effet, un arbre AVL de hauteur h possède au moins autant de noeuds qu'un arbre de Fibonacci de hauteur h (et au plus 2^h), ie. $n \geq c\varphi^h$. Et par le même raisonnement que pour les arbres de Fibonacci, on en déduit que la hauteur est $O(\log(n))$.

Question 11

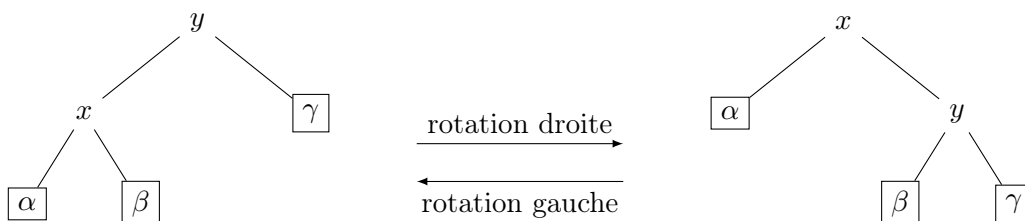
Écrire un algorithme $EST_AVL(a)$ qui détermine si l'arbre a est AVL.

SOLUTION

```
function EST_AVL(a)
  if a == NIL then
    return true
  else
    return EST_AVL(a.gauche) and EST_AVL(a.droite) and |a.gauche.hauteur - a.droite.hauteur| ≤ 1
```

Question 12

Soit a un arbre binaire de recherche et y l'un de ses noeuds. Une *rotation droite* autour de y consiste à faire descendre ce noeud et à faire remonter son fils gauche x sans invalider l'ordre des éléments:



L'opération inverse s'appelle une *rotation gauche* autour du sommet x .

Montrer qu'une rotation préserve la structure d'arbre binaire de recherche et peut se réaliser à coût constant lorsqu'elle est effectuée autour de la racine.

Considérons une rotation droite et notons a , b et c des éléments quelconques des ABR α , β et γ . Par hypothèse, $a \leq x \leq b$ et $b \leq y \leq c$ ce qui équivaut à $b \leq y \leq c$ et $a \leq x \leq b$ et ce qui traduit que l'arbre obtenu par rotation droite est toujours un ABR.

Pour montrer que ces rotations sont effectuées à coût constant, il suffit d'en donner les algorithmes:

```

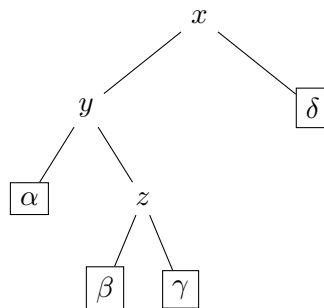
function ROTATION_DROITE( $y$ )
     $x \leftarrow y.gauche$ 
     $\beta \leftarrow x.droite$ 
     $y.gauche \leftarrow \beta$ 
     $x.droite \leftarrow y$ 
     $y.hauteur \leftarrow 1 + \max(\beta.hauteur, y.droite.hauteur)$ 
     $x.hauteur \leftarrow 1 + \max(x.gauche.hauteur, y.hauteur)$ 
    return  $x$ 

function ROTATION_GAUCHE( $x$ )
     $y \leftarrow x.droite$ 
     $\beta \leftarrow y.gauche$ 
     $x.droite \leftarrow \beta$ 
     $y.gauche \leftarrow x$ 
     $x.hauteur \leftarrow 1 + \max(\beta.hauteur, x.gauche.hauteur)$ 
     $y.hauteur \leftarrow 1 + \max(y.droite.hauteur, x.hauteur)$ 
    return  $y$ 

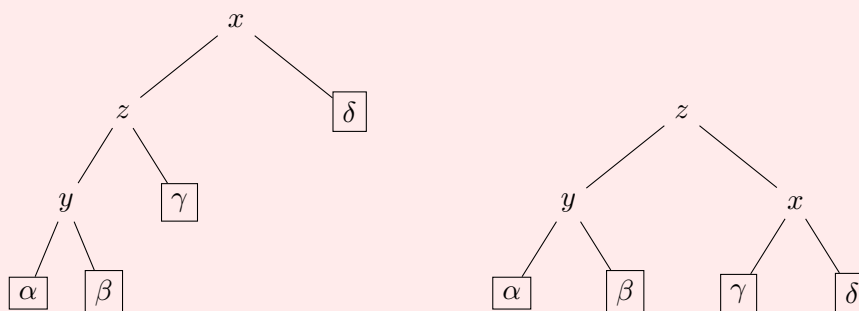
```

Question 13

Appliquer une rotation gauche autour de y puis une rotation droite autour de x dans l'arbre suivant:



Les 2 rotations successives sont:



Question 14

On considère désormais un arbre binaire de recherche qui respecte la condition AVL, à savoir que le déséquilibre de chaque nœud est égal à -1, 0 ou 1, et on considère une insertion qui a conduit à la création d'une

feuille f . Seuls les ancêtres de f ont vu leur déséquilibre modifié; on suppose que l'un au moins ne respecte plus la condition AVL et on note y le premier de ceux-là. Montrer que $deseq(y) = \pm 2$.

SOLUTION

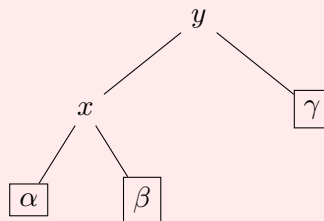
Le déséquilibre initial de y est égal à -1 , 0 ou 1 , et l'apparition d'une feuille parmi ses descendants ne peut modifier son déséquilibre que d'au plus une unité, donc $deseq(y) = \pm 2$.

Question 15

Sans perte de généralité, on suppose que $deseq(y) = 2$, et on note x le fils gauche de y . Montrer que si $deseq(x) = 0$ ou $deseq(x) = 1$, une rotation suffit pour retrouver la condition AVL.

SOLUTION

Si on suppose $deseq(y) = 2$ (et donc que l'insertion a été effectuée dans x le fils gauche de y), la situation peut être représentée ci-dessous:



avec $h(\gamma) = \max(h(\alpha), h(\beta)) - 1$.

Si $deseq(x) = 0$ alors $h(\alpha) = h(\beta)$ et $h(\gamma) = h(\beta) - 1$, et la rotation droite autour de y conduit aux nouvelles valeurs du déséquilibre : $deseq(x) = h(\alpha) - (h(\beta) + 1) = -1$ et $deseq(y) = h(\beta) - h(\gamma) = 1$ donc la condition AVL est de nouveau respectée.

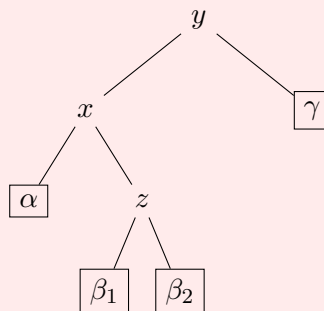
Si $deseq(x) = 1$ alors $h(\alpha) = h(\beta) + 1$ et $h(\gamma) = h(\beta)$, et la rotation droite autour de y conduit aux nouvelles valeurs du déséquilibre : $deseq(x) = h(\alpha) - (h(\beta) + 1) = 0$ et $deseq(y) = h(\beta) - h(\gamma) = 0$ donc la condition AVL est de nouveau respectée.

Question 16

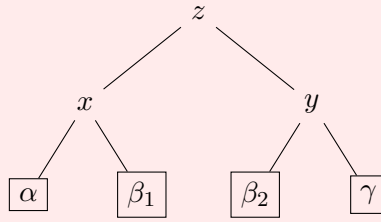
Montrer que $deseq(x) = -1$, deux rotations suffisent pour retrouver la condition AVL.

SOLUTION

Si $deseq(x) = -1$ alors $h(\beta) = h(\alpha) + 1$ et $h(\gamma) = h(\alpha)$. Puisque $h(\beta) > 0$, β n'est pas l'arbre vide ; notons z sa racine, β_1 et β_2 ses fils gauche et droit. La situation est alors la suivante:



avec $\max(h(\beta_1), h(\beta_2)) = h(\alpha) = h(\gamma)$. La question 13 a montré qu'en deux rotations il était possible d'obtenir l'arbre ci-dessous:



z étant équilibré on a $h(\beta_1) - h(\beta_2) = -1, 0$ ou 1 donc les nouvelles valeurs du déséquilibre sont : $deseq(x) = 0$ ou 1 , $deseq(y) = 0$ ou -1 , $deseq(z) = 0$, donc la condition AVL est de nouveau respectée. Il reste à remonter dans la branche menant à la racine et à réitérer éventuellement le processus pour rééquilibrer un arbre AVL après une insertion en un coût $O(h(a))$.

Question 17

Écrire l'algorithme `INSERER_AVL(a, v)` qui insère la valeur v dans l'arbre AVL a .

SOLUTION

```

function INSERER( $a, v$ )
  if  $a == NIL$  then
    return feuille( $v$ )
  if  $v < a.cle$  then
     $a.gauche \leftarrow \text{Inserer}(a.gauche, v)$ 
    if  $deseq(a) == 2$  then
      if  $deseq(a.gauche) == -1$  then
         $a.gauche \leftarrow \text{rotation\_gauche}(a.gauche)$ 
       $result \leftarrow \text{rotation\_droite}(a)$ 
    else
       $result \leftarrow a$ 
  else
     $a.droite \leftarrow \text{Inserer}(a.droite, v)$ 
    if  $deseq(a) == -2$  then
      if  $deseq(a.droite) == 1$  then
         $a.droite \leftarrow \text{rotation\_droite}(a.droite)$ 
       $result \leftarrow \text{rotation\_gauche}(a)$ 
    else
       $result \leftarrow a$ 
   $result.hauteur \leftarrow 1 + \max(a.gauche.hauteur, a.droite.hauteur)$ 
  return  $result$ 

```

Attention: remarquez que dans le cas où $deseq(a) == -2$ on teste si $deseq(a.droite) == 1$ (et non -1 comme dans le cas où $deseq(a) == 2$)